

IRE Assignment Design Note

Iman Kalyan Chakraborty - 2026801003

August 21, 2026

1 Introduction

This assignment contains an implementation of a complete lexical and semantic retrieval pipeline news recommendation using two production-level, real-world datasets, EB-NeRD and MIND. Both datasets have significant differences, including their storage schema. The datasets have multiple variants:

- EB-NeRD has demo, small and large variants
- MIND has small and large variants

The small and demo variants have been used for experimentation and the large variants have been used for the Codabench submissions.

Broadly, the pipeline contains the following steps:

1. **Unified Schema Conversion:** Since the raw data from both datasets is different from each other, they need to be unified into a common format so that the common format can be used for training.
2. **Lexical Candidate Generation:** We have used a simple from-scratch implementation of BM25 instead of the popular `rank_bm25` package that was freely available because the from scratch implementation was faster.
3. **Semantic Candidate Generation:** This phase requires computing the embeddings for the datasets using `paraphrase-xlm-r-multilingual-v1`, an XLM-RoBERTa-based 768-dimensional embeddings generator, as suggested per the rules of the assignment. The multilingual variant was selected because EB-NeRD contains Danish text while MIND contains English.
4. **Evaluation:** The model has been evaluated against all metrics provided in the assignment, namely AUC, MRR, nDCG@k, diversity, novelty and coverage.

The remainder of the text contains the design decisions behind each stage and the corresponding experimental observations.

2 Design

2.1 Preliminaries

2.1.1 BM-25

BM-25 (short for Best Match 25) is an old and reliable ranking algorithm that ranks a document d based on a query q . This rank tells us how relevant d is in the context of q . It does so using 3 main components:

1. **Term Frequency:** $TF(t, d) = \frac{raw_freq(t, d)}{raw_freq(t, d) + k_1 * (1 - b + b * \frac{|d|}{avgdl})}$ where k_1 controls how quickly extra occurrences of a term stop adding to its score, b controls how much a document's length is penalized, $|d|$ is the number of tokens in document d , and $avgdl$ is the average document length over the whole corpus.
2. **Inverse Document Frequency:** A term that appears in most documents is not a useful signal, so it should count for less. The non-negative variant is used here instead of the classic Robertson/Sparck-

Jones form, since the classic form can go negative once a term appears in more than half the corpus:

$$IDF(t) = \log \left(\frac{N - df(t) + 0.5}{df(t) + 0.5} + 1 \right)$$

where N is the number of documents in the corpus and $df(t)$ is the number of documents that contain term t . The $+1$ inside the log keeps $IDF(t) \geq 0$ for every term.

3. **Document Length Normalization:** This is already folded into the Term Frequency formula above, through the $(1 - b + b \cdot \frac{|d|}{avgdl})$ term. A document longer than the corpus average is penalized, since a term is more likely to appear in it by chance alone. Setting $b = 0$ turns this normalization off completely, and $b = 1$ applies it in full.
4. **Final Score:** The score of document d against query q is the sum, over every term in the query, of that term's IDF weighted by its (rescaled) TF in the document:

$$score(q, d) = \sum_{t \in q} IDF(t) \cdot (k_1 + 1) \cdot TF(t, d)$$

k_1 and b are set to the standard values 1.5 and 0.75 throughout this project.

Each subsection below covers one question from the assignment: what was built, and the key choice behind it. The full reasoning, including alternatives that were considered and rejected, is in `SPEC.md`.

2.2 Q1: Reproducible Data Pipeline

One shared schema is used for both datasets, but the datasets are never merged. EB-NeRD demo, EB-NeRD small, and MIND-small are each built into their own output directory by the same pipeline code, since they share no users, articles, or IDs. Every ID is namespaced by dataset (e.g. `ebnerd_123`, `mind_N55528`) so a bug can never silently join rows across datasets. The schema has three tables: **articles**, **behaviors** (one row per impression), and **history** (per-user pre-window click history).

The split is never random. Each dataset ships a provider train split followed by a provider dev/validation split, in that time order. The provider's dev/validation split becomes the held-out **test** set, and **val** is carved from the last day of the provider's **train** split, by time. Two leakage checks run at build time: EB-NeRD's history timestamps must precede every impression for that user, and MIND's single history string (embedded per row, not a separate table) must be identical across all of a user's rows, proving it is a fixed pre-window snapshot. The pipeline rebuilds from raw files with one command, `uv run python build_pipeline.py`; every step has a test cell that must pass, so a broken build fails loudly instead of silently writing wrong data.

2.3 Q2: Lexical Candidate Generation (BM25)

BM25 is implemented from scratch in `bm25.py` instead of the `rank_bm25` package, purely for speed. `rank_bm25`'s scoring does a plain dictionary lookup across every document for each query term, measured at close to 3 seconds per query on MIND's 65,238-article corpus, which extrapolates to over 40 hours at the scale needed here (once per val/test user). The from-scratch version builds a real postings-list index (term to matching document indices and their term frequencies), scores it with a vectorized numpy operation over only the documents containing each query term, and picks the top-K with `np.argpartition`. This measured at about 5ms per query, roughly a thousand times faster.

A user's query is the titles of their most recent 20 clicks, concatenated. This window depends only on a user's fixed history, so the top-200 scores are computed once per user and cached; `recall@50` and `recall@100` are read off as prefixes of that same list, instead of scoring the same query three times. `Recall@K` is a fraction, $|clicked \cap top-K|/|clicked|$, not a plain hit or miss, since one impression can have more than one clicked article.

2.4 Q3: Semantic Candidate Generation (Embeddings)

One embedding model, `paraphrase-xlm-r-multilingual-v1`, is used for both datasets instead of EB-NeRD’s own provided embeddings, so that Danish and English article text lands in the same vector space. MIND has no article embeddings of its own, only unrelated knowledge-graph entity vectors, so a single shared model keeps both datasets comparable for Section 3.1.

Embeddings are computed once, on a Kaggle GPU notebook, and saved as parquet files, so the local environment never installs `sentence-transformers` or `torch`; everything downstream (mean-pooling, retrieval, evaluation) only needs numpy, pandas, and pyarrow. Retrieval is a brute-force batched matrix multiply plus `argpartition`, not FAISS: at this corpus size (up to about 65,000 articles), the full MIND val/test population retrieves in well under two minutes, so an ANN library would only add a dependency to solve a problem numpy already solves fast enough. Users are still batched in groups of a few thousand, since the full similarity matrix at MIND’s scale would take roughly 17GB of memory at once. A user’s representation is the mean of their last 20 clicked articles’ embeddings, the same window Q2 uses, so Section 3.1’s comparison only varies the scoring method, not the input.

2.5 Q4: Offline Evaluation Harness

AUC, MRR, and nDCG need a full ranking over candidates with known labels. The only such set per impression is `article_ids_inview`, what the user was actually shown, not the whole catalog. This differs from Q2/Q3’s recall@K, which asks whether a method can find the clicked article at all among everything; these metrics ask whether it ranks the clicked article near the top among what was shown. Q4 cannot reuse Q2/Q3’s saved top-200 lists for this, since MIND’s recall@200 is only a few percent and the clicked article is usually not in it. Instead a re-scoring adapter, `score_inview`, computes each method’s score for exactly the items in one impression’s inview set, on the fly.

Beyond-accuracy metrics: intra-list diversity is one minus the average share of same-category pairs in the top-10 re-ranked list; novelty is the negative log of an item’s train-split click share, add-one smoothed for items never clicked in train; coverage is the fraction of the catalog appearing anywhere in the saved top-200 lists across all evaluated users. Cold-start vs. warm users is the required slice, and head vs. tail articles (top 20% by train click count vs. the rest) is added as a second one. Every metric carries a 95% bootstrap confidence interval over 1,000 resamples of impressions, fast enough to run in well under a second even for MIND’s roughly 70,000-impression test split.

2.6 Q5: Codabench Submission

The submission format, one line per impression as `{impression_id} [rank_1,...,rank_n]` in the original row order, was confirmed directly from MIND’s Submission Guidelines page and matches both the EB-NeRD starter repository and the reference MIND example code. Prediction files were generated for all three dataset tracks with `generate_predictions.py`, reusing Q4’s `score_inview` adapters.

Both real submissions made so far have failed on Codabench, and this is not yet fixed. MIND failed with a candidate-set size mismatch, EB-NeRD with an impression ID that does not exist locally. Both trace to the same cause: each competition grades against a separate, larger, hidden test file (`MINDlarge_test`, `ebnerd_testset.zip`), not the provider dev/validation split used locally as the test set. Download scripts for both exist but have not been run yet. Leaderboard screenshots will be added once a real submission scores.

3 Discussion

3.1 Lexical vs. Semantic Retrieval

Table 1 shows recall@200 on every dataset track and split.

Dataset	Split	BM25	Embedding
EB-NeRD demo	val	3.51%	2.85%
EB-NeRD demo	test	2.85%	2.61%
EB-NeRD small	val	2.10%	1.48%
EB-NeRD small	test	1.82%	1.49%
MIND	val	3.40%	2.39%
MIND	test	1.61%	1.92%

Table 1: Recall@200, lexical (BM25) vs. semantic (embedding) candidate generation.

BM25 leads almost everywhere, except MIND’s test split, where embeddings overtake it at both K=50 (0.65% vs. 0.45%) and K=200. BM25 also covers more of each catalog: coverage@200 is 92–99.5% for BM25 across the three tracks, but only 56–96.7% for embeddings, since mean-pooled vectors tend to concentrate retrieval on a narrower, similar-feeling slice of the catalog.

This reverses under Q4’s ranking metrics, which score a method only on the small inview set actually shown to a user. On MIND’s test split, embeddings reach an AUC of 0.595 against BM25’s 0.556; on EB-NeRD small, 0.550 against 0.501. So embeddings are worse at finding the clicked article among the whole catalog, but better at ranking it once the candidate set is already narrowed to what the user saw. Recall@K and AUC answer different questions and do not have to agree.

3.2 Dataset Differences

- **Language:** EB-NeRD is Danish, MIND is English. One multilingual embedding model and one Unicode-aware word tokenizer handle both, with no dataset-specific code.
- **Cold-start:** EB-NeRD demo and small have zero cold-start users in val/test. MIND has 1,770 out of 65,173.
- **Popularity skew:** MIND’s top 20% of ever-clicked articles account for 90.2% of train clicks; EB-NeRD demo is flatter (49.2%), EB-NeRD small in between (69.0%). About 90–92% of each catalog is never clicked in train, in all three tracks.
- **Text length:** MIND’s average tokenized title+abstract (47.2 tokens) is close to double EB-NeRD’s (25.1–25.8), part of why MIND’s BM25 recall runs lower.

3.3 Where This Breaks at 10x Scale

- **Per-user loops:** Q1’s MIND history build and Q2/Q3’s retrieval caching loop once per user. At 10x the users these need to move to fully vectorized batch code, not just the embedding step, which is already batched.
- **The similarity matrix:** embedding similarity for all MIND users against all MIND articles at once is already close to 17GB, computed only in batches. At 10x scale this needs a real ANN index such as FAISS, or a much smaller batch size.
- **Single-machine I/O:** the pipeline reads and writes parquet with pandas on one machine, fine at hundreds of thousands of impressions. At 10x scale RAM, not compute, becomes the limit, and a real feature store (a database or distributed format) is needed.
- **BM25 scales fine as is:** the postings-list index is already sub-millisecond per query, so it would likely survive 10x scale without a redesign, only needing more batching in the surrounding loop.